
PowerInfer on Consumer GPUs: Reproducing Sparse LLM Inference and Analyzing Prompt-Dependent Behavior

Jamal Mapp
University of Central Florida
jamaltmapp@gmail.com

Shawn McChargue
University of Central Florida
smcchargs@gmail.com

Abstract

Large Language Models (LLMs) are often limited by memory bandwidth rather than compute, making inference inefficient on consumer-grade GPUs. PowerInfer introduces a sparsity-aware approach that exploits the observation that only a subset of neurons activate during each token generation step. In this work, we reproduce PowerInfer’s methodology on consumer hardware and evaluate its performance relative to dense inference approaches. Our experiments demonstrate up to a 60% improvement in token generation speed while maintaining similar memory usage. Additionally, we analyze how neuron activation patterns vary across different prompt types, revealing that inference performance is dependent on input complexity. These findings validate the effectiveness of sparse inference and highlight opportunities for further optimization.

1 Introduction

Large Language Models (LLMs) have achieved state-of-the-art performance across a wide range of tasks, but their deployment remains constrained by high computational and memory requirements. A key bottleneck in LLM inference is memory bandwidth: for each generated token, large model weights must be repeatedly loaded from memory, leading to inefficiencies, particularly on consumer-grade GPUs with limited VRAM.

PowerInfer addresses this challenge by leveraging sparsity in neuron activation. Rather than computing all neurons in the feed-forward network, it selectively computes only those that are likely to activate. This reduces memory movement and improves efficiency.

In this project, we reproduce PowerInfer’s approach on consumer hardware and evaluate its performance relative to traditional dense inference methods. Additionally, we extend the analysis by examining how different input prompt types influence neuron activation and inference performance.

2 Related Work

Several approaches have been proposed to improve LLM inference efficiency. Traditional frameworks such as *llama.cpp* focus on enabling model execution on consumer hardware through techniques such as quantization and layer offloading, but they retain dense computation.

Systems such as *vLLM* optimize throughput through memory-efficient attention mechanisms like PagedAttention, but still rely on dense computation of model weights.

PowerInfer differs by exploiting sparsity in neuron activation, reducing unnecessary computation and memory transfers. This approach aligns with broader research trends in sparse neural networks and efficient model serving.

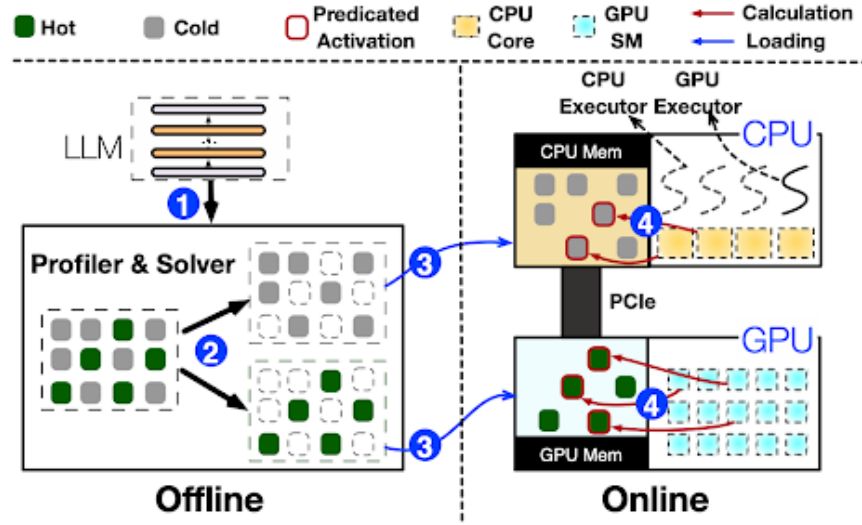


Figure 1: PowerInfer inference pipeline. The system predicts active neurons and splits computation between GPU (hot neurons) and CPU (cold neurons).

3 Method

3.1 PowerInfer Overview

PowerInfer is based on the observation that neuron activation in LLMs exhibits locality and follows a power-law distribution. A small subset of neurons, referred to as *hot neurons*, are activated frequently, while the majority, *cold neurons*, are rarely used.

During inference, PowerInfer predicts which neurons will activate at each token step. Hot neurons are placed on the GPU, while cold neurons remain on the CPU. By computing only the active subset, the system reduces memory bandwidth usage and accelerates inference.

3.2 Implementation

We implemented an experimental pipeline using a Python script that automates model execution and extracts performance metrics from PowerInfer logs. The script utilizes subprocess calls to execute PowerInfer binaries and parses output logs using regular expressions.

The pipeline records:

- Token generation speed (tokens per second)
- Prompt processing speed
- VRAM usage
- Feed-forward network (FFN) offloading size

This approach enables reproducible benchmarking across multiple models and configurations.

4 Experiments

4.1 Setup

All experiments were conducted on a consumer-grade system with the following specifications:

- GPU: NVIDIA RTX 5060 Ti (8GB VRAM)
- RAM: 32GB

Table 1: Models evaluated in our experiments and their approximate memory requirements.

Model	Quantization	Size (GB)	Notes
ReluLLaMA-7B	f16	~14.7	Small Model
ReluLLaMA-13B	Q4	~8.1	Barely Fits Model
ReluFalcon-40B	Q4	~24.8	Oversized Model

- CPU: Intel Core Ultra 7

We evaluated the following models:

- ReluLLaMA-7B (PowerInfer and baseline)
- ReluLLaMA-13B
- ReluFalcon-40B

To ensure consistency, all experiments used:

- Identical prompts
- Output length of 128 tokens
- Temperature = 0.0
- Repeat penalty = 1.1

4.2 Results

In a direct comparison between PowerInfer and a baseline dense implementation, our results indicate:

- PowerInfer: 3.37 tokens/sec
- Baseline: 2.1 tokens/sec

This corresponds to approximately a 60% improvement in generation speed, while maintaining similar VRAM usage.

For larger models, performance varied:

- 7B model achieved strong speed improvements
- 13B model exhibited performance degradation due to CPU bottlenecks
- 40B model exceeded hardware constraints

4.3 Prompt Disparity Analysis

We evaluated model performance across different prompt types:

- Natural language text generation
- Code generation
- Mathematical reasoning

Results indicate that inference speed varies significantly depending on the prompt type. Simpler, high-sparsity tasks achieved higher throughput, while complex reasoning tasks triggered more diverse neuron activation patterns, reducing performance.

These findings suggest that neuron sparsity is input-dependent, creating opportunities for adaptive inference optimization.

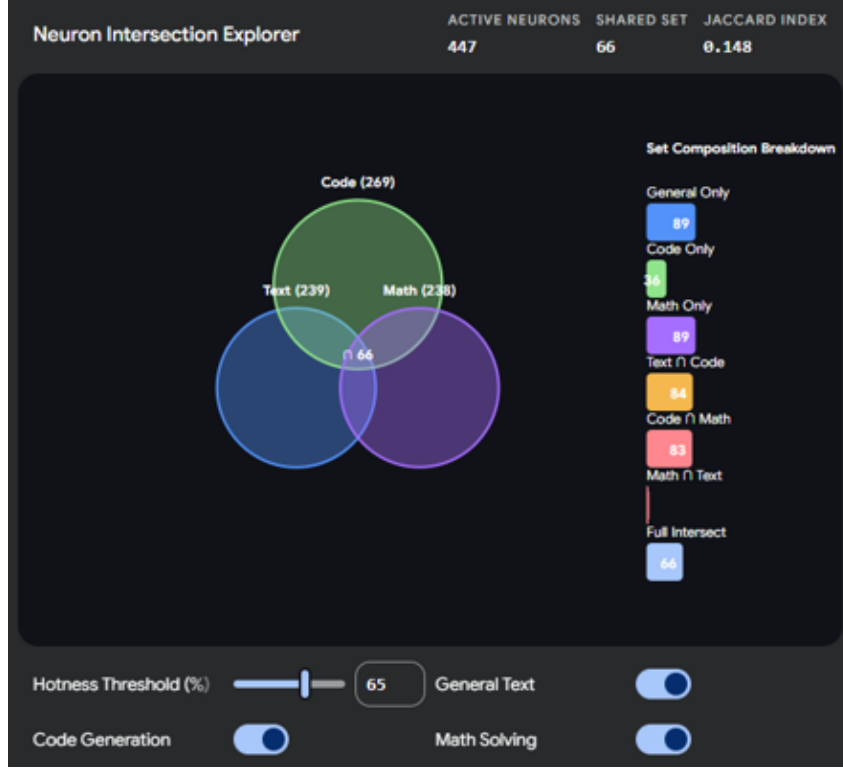


Figure 2: Inference performance varies across prompt types, suggesting input-dependent neuron activation patterns.

5 Analysis and Discussion

The results confirm that PowerInfer’s effectiveness stems from exploiting sparsity in neuron activation. By minimizing memory transfers and focusing computation on frequently activated neurons, it improves efficiency on consumer hardware.

However, performance gains are not uniform. For larger models, CPU-to-memory bandwidth becomes a limiting factor, as cold neuron computation cannot keep pace with GPU execution.

Additionally, prompt-dependent behavior suggests that sparsity is dynamic. Tasks requiring diverse reasoning activate more neurons, reducing the effectiveness of selective computation.

Limitations of this study include hardware constraints and reliance on a single experimental setup. Future work should explore more powerful systems and alternative sparsity strategies.

6 Conclusion

This work reproduces PowerInfer’s sparse inference approach and demonstrates its effectiveness on consumer-grade GPUs. Our experiments show significant improvements in token generation speed while maintaining similar memory usage.

Furthermore, we highlight that inference performance depends on input characteristics, suggesting opportunities for adaptive optimization strategies. These findings reinforce the potential of sparsity-based methods for efficient LLM deployment.

Contribution Statement

- Jamal Mapp contributed to methodology explanation, experimental analysis, and report writing.
- Shawn McChargue contributed to experiment implementation, code development, and repository management.

A Additional Results

Additional logs, benchmark outputs, and experimental artifacts are included in the accompanying GitHub repository for reproducibility:

<https://github.com/smcchargs/CAP-6614-PowerInferMetrics>